

Ключові метрики розробки ПЗ

ЛЕКЦІЯ 10

Наскільки продуктивно працює команда розробників? Коли планувати реліз? Чи буде продукт справді корисним для користувачів? Щоб відповіді на ці питання існують метрики розробки програмного забезпечення.

на різних етапах важливо контролювати відповідні метрики.

- Під час розробки — **менеджерські метрики**,
- Перед релізом — **технічні метрики**,
- Після релізу — **користувацькі метрики**, щоб зрозуміти, як аудиторія взаємодіє з продуктом.

Існують сотні різних метрик.

Розглянемо ключові показники (метрики)

МЕНЕДЖЕРСЬКІ МЕТРИКИ

Успіх усього проєкту залежить від ефективності процесу розробки. Бюджет, терміни, пріоритетні завдання — усе це слід обирати та коригувати на основі конкретних цифр. Менеджерські метрики допомагають компаніям з розробки програмного забезпечення оцінювати роботу команди.

Velocity

- Швидкість команди у виконанні завдань за спринт.
- Допомогає прогнозувати терміни релізів і планувати навантаження.
- Дозволяє планувати обсяги майбутніх спринтів.

Ця метрика розробки програмного забезпечення показує, який обсяг роботи команда здатна виконати за спринт. Для обчислення показника продуктивності в продуктивних командах використовують сторі-поїнти (story points) — за їх допомогою кожному завданню в беклозі призначають вагу залежно від складності, замість сторі-поїнтів оцінювання може відбуватися в годинах. Сума всіх сторі-поїнтів або всіх годин за спринт — це і є продуктивність.

- ☐ Метрику використовують для того, щоб:
- зрозуміти, чи команда справляється з темпами проєкту, або ж потрібно наймати додаткових працівників;
- планувати кількість завдань у наступних спринтах;
- відстежувати, як ті чи інші зміни в робочому процесі впливають на продуктивність;
- планувати час релізу продукту, виходячи з реальних можливостей розробки.

Burn Rate

Це одна з найважливіших менеджерських метрик у розробці програмного забезпечення: діаграма «згоряння» завдань показує, скільки роботи вже виконано, а скільки ще залишилося. Метрику можна переглянути за:

конкретний спринт — це називається **sprint burndown chart** або діаграма згоряння завдань за спринт;

увесь проєкт — це називається **release burndown chart** або діаграма згоряння завдань до релізу.

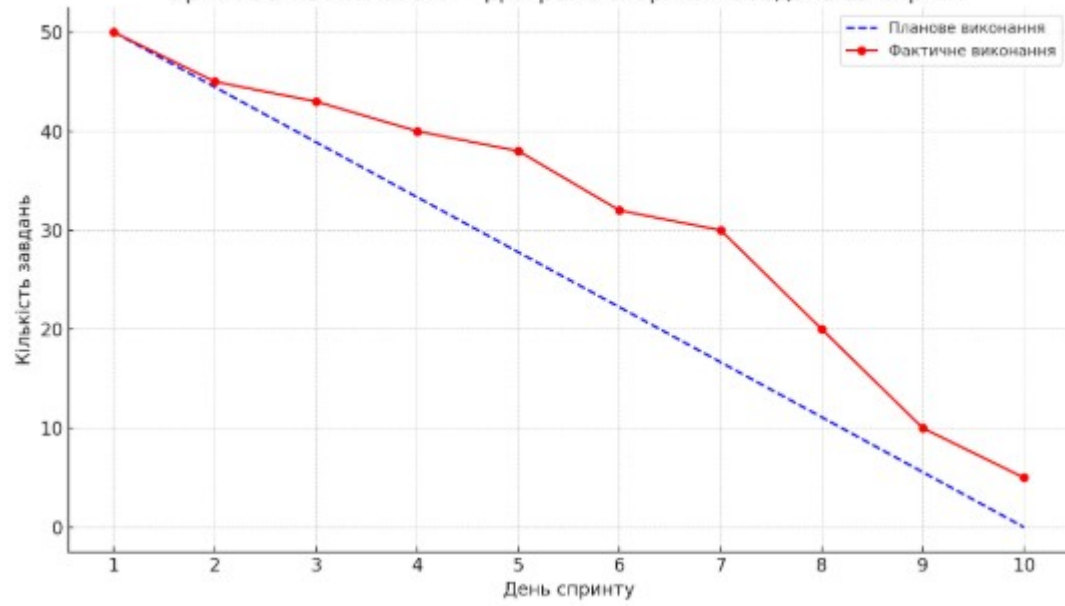
Графік виглядає як криві, що спускаються вниз: вони відображають динаміку виконання завдань.

По осі X позначають кількість днів до завершення спринту або релізу.

По осі Y — кількість завдань.

Одна з ліній графіка показує, з якою швидкістю планувалося виконання роботи. Друга лінія — як робота виконується насправді.

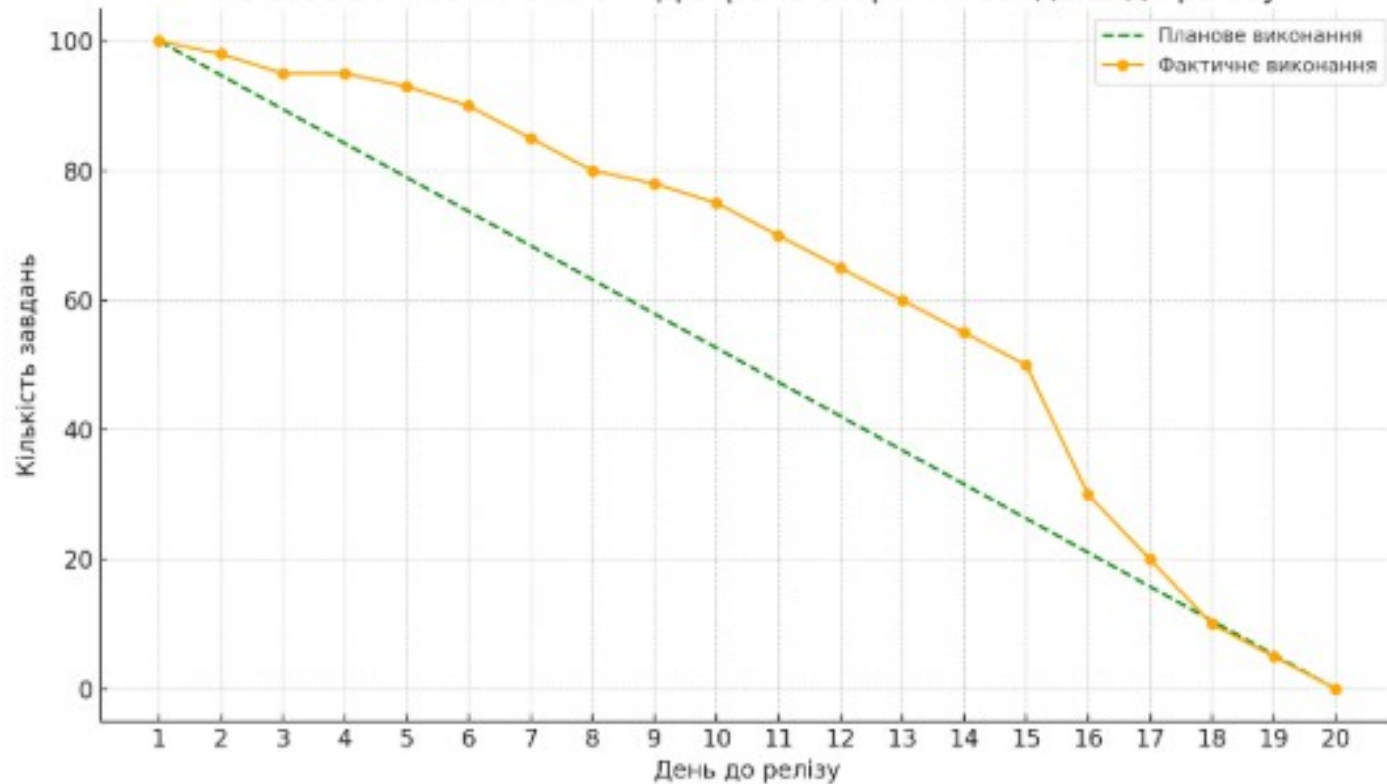
Sprint Burndown Chart — Діаграма згоряння завдань за спринт



Якщо діагональ, що відображає реальну роботу, має крутіший нахил, це означає, що робота виконується швидше, ніж планувалося. Якщо ж вона більш полого — справа просувається повільніше. Також графік фактичної роботи може бути сильно викривленим. Наприклад, спочатку йти горизонтально (коли завдання не виконуються взагалі), а потім різко опускатися вниз (якщо команда, що надає послуги з розробки ПЗ, працює в ударному темпі).

- □ За діаграмою згоряння завдань можна визначити:
- чи реально закрити всі завдання за спринт;
- чи встигне команда зробити реліз у заплановані строки;
- на якому етапі знаходиться проєкт зараз;
- наскільки планомірно працює команда;
- наскільки правильно розраховується час на виконання завдань.

Release Burndown Chart — Діаграма згорання завдань до релізу



Flow Efficiency

У розробці програмного забезпечення на замовлення існують періоди активної роботи та час очікування: коли для того, щоб розпочати завдання, не вистачає інформації або ресурсів. Мінімізувати простої — одне із завдань менеджера. Співвідношення часу активної роботи до загального часу, включаючи простої, називають **ефективністю потоку**.

Наприклад, якщо над завданням працювали 3 дні, а ще 2 воно «висіло» в очікуванні, коли розробник звільниться для нього, формула ефективності потоку буде: $3/(2+3) * 100\%$.

□ Ця метрика розробки програмного забезпечення слугує для того, щоб:

- оцінити, як довго простоюють завдання і чи є це проблемою для проєкту;
- оптимізувати аутсорсинг розробки програмного забезпечення та контролювати час роботи.

Lead Time

Час від створення задачі до її виконання

- Метрика вимірює час, який проходить від початку роботи над завданням (або запитом) до його завершення. Вона відображає, скільки часу знадобилося для того, щоб завдання було виконано, починаючи з моменту, коли воно було поставлене в роботу, до моменту, коли воно готове до використання.
1. **Вимірює ефективність процесу** — показує, наскільки швидко команда здатна завершити завдання.
 2. **Знижує час очікування** — допомагає виявити затримки або проблеми, що виникають в процесі розробки.
 3. **Покращує прогнозування** — дає змогу краще прогнозувати терміни виконання завдань.

Lead Time можна розрахувати як різницю між часом початку роботи над завданням та часом його завершення.

Cycle Time

Це показник, який вимірює час, що проходить від початку роботи над завданням (наприклад, від початку розробки функції або від прийняття завдання в роботу) до його завершення (коли завдання готове, прийняте або поставлене в продакшн)

Як розраховується Cycle Time:

1. **Початок:** Точка часу, коли завдання (або робота) було розпочато, прийнято в роботу або іншим чином почала рухатись.
2. **Кінець:** Коли завдання завершено, прийняте або знаходиться на необхідному етапі, наприклад, в продакшн.

Застосування:

- **Продуктивність команди:** Аналізуючи Cycle Time, можна зрозуміти, скільки часу команда витрачає на виконання завдань. Це допомагає виявити вузькі місця в процесі.

- **Прогнозованість:** Зниження часу циклу дозволяє покращити прогнозованість завершення завдань, що допомагає команді краще планувати свої дії.

- **Оцінка ефективності процесу:** За допомогою Cycle Time можна аналізувати, як різні етапи роботи впливають на загальний процес. Це може допомогти в прийнятті рішень щодо покращення робочого процесу.

Deployment Frequency

- Частота розгортання змін у продуктивне середовище.
- Часті релізи підвищують гнучкість і швидкість бізнесу.

Вона вимірює, як часто команда виконує розгортання змін у продакшн або доставляє їх користувачам.

□ Що вимірює метрика: Кількість розгортань (деплоїв) у продакшн за певний період часу (наприклад, за день, тиждень або місяць).

□ Мета: Оцінити, наскільки швидко і регулярно команда доставляє корисні зміни користувачам — нові функції, виправлення помилок, покращення продуктивності тощо.

Work in Progress (WIP — Задачі в роботі)

Що вимірює: кількість задач, які виконуються одночасно.

Для чого потрібна:

Виявлення перевантаження команди.

Забезпечення фокусу на завершенні задач, а не на багатозадачності.

Employee Satisfaction (Задоволення команди)

Що вимірює: рівень задоволення роботою, атмосферою, процесами в команді.

Як оцінити:

Анонімні опитування.

Регулярні one-on-one зустрічі.

Індекси eNPS (Employee Net Promoter Score).

Для чого потрібна:

- Зниження плинності кадрів.
- Побудова здорової культури в команді.
- Виявлення потенційного вигорання або конфліктів.

Budget Adherence (Дотримання бюджету)

Що вимірює: наскільки фактичні витрати збігаються з запланованим бюджетом.

Для чого потрібна:

- Контроль витрат на проєкт.
- Виявлення перевитрат або неефективності.

Як використовувати:

- Регулярно порівнювати плановий і фактичний бюджет.
- Аналізувати причини відхилення — зокрема перевитрати на зміну вимог або затримки.

Технічні метрики

Показники, які використовуються для **оцінки якості, ефективності та стабільності програмного коду**, а також для підтримки високого технічного рівня проєкту.

Bug Rate

Середня кількість багів за спринт

- Кількість знайдених помилок на певний обсяг роботи.
- Показник якості розробки та тестування.
- Баги при розробці ПЗ виникають неминуче. Щоб їх контролювати, можна підраховувати кількість у кожному спринті. На прикладі кількох спринтів можна визначити нормальний показник для конкретного проєкту і в подальшому орієнтуватися на нього. А суттєві відхилення від середнього числа — привід задуматися.
- ☐ Ось що можна побачити за кількістю багів за спринт:
- Якщо багів занадто багато, значить, у розробці щось пішло не так: потрібно виявити причину систематичних помилок.
- Якщо багів занадто мало, це теж привід насторожитися — можливо, тестування було недостатньо ретельним.

Test Coverage

Щільність покриття програмного коду тестами — це відсоток коду, який було перевірено під час тестування. Наприклад, якщо цей показник становить 78%, це означає, що 78% усього написаного програмістами коду було охоплено тестами.

□ Для спеціалістів компаній, що займаються розробкою ПЗ, ця метрика допомагає зрозуміти: чи достатньо тестів було проведено; наскільки великий ризик, що в програмі залишилися непомічені баги, які можуть потрапити у реліз.

Code Churn

- Процент коду, що змінюється або видаляється після першого коміту.
- Вона показує, як часто й інтенсивно редагується код після початкового коміту.
- ☐ Що вимірює: Code Churn відображає: скільки коду було змінено або видалено після додавання; частоту та обсяг змін у файлах або модулях; стабільність та впевненість розробників у внесених змінах.
- Приклад: Розробник додає 100 рядків коду. Через 2 дні змінює 60 з них. Code Churn = 60%, тобто 60% доданого коду було змінено майже одразу.
- ☐ Для чого використовується: Виявляти нестабільні ділянки коду, які часто змінюються. Аналізувати ефективність планування: часті переписування можуть вказувати на неясні вимоги. Визначати ризики перед релізом — велика "турбулентність" сигналізує про потенційні помилки. Високий показник може свідчити про нестабільність.

Escaped Defects

- Ця метрика **показує** кількість багів (помилкок), які не були виявлені на етапі тестування і потрапили у продакшн, тобто стали видимими для кінцевих користувачів.
- **Що вимірює:** Ця метрика показує, наскільки ефективно працює внутрішнє тестування (QA):
 - Скільки дефектів не було знайдено вчасно;
 - Яка частка помилок доходить до користувача;
 - Наскільки надійний і якісний процес контролю якості.

Приклад: За місяць було виявлено 100 багів. 20 з них повідомили користувачі після релізу. Escaped Defects = 20, або 20% — це ті, що «втекли» з процесу тестування.

□ **Навіщо потрібна:** Виявляти слабкі місця в QA-процесі; Покращувати автоматизоване та ручне тестування; Оцінювати ризики для користувача досвіду.

Technical Debt

- Обсяг неоптимізованого коду або рішень.

Це метрика, яка відображає обсяг тимчасових рішень, компромісів або недоліків у коді, які виникають у процесі розробки і потребують виправлення в майбутньому.

Цей термін нагадує фінансову заборгованість: що довше її ігнорувати, то дорожче «відсотки» — підтримка, баги, складність змін.

□ Навіщо потрібна ця метрика: - Впливає на швидкість розробки в майбутньому.

Оцінити довгострокові ризики для підтримки та розвитку продукту;

Планувати час на рефакторинг і «погашення» боргу;

Встановлювати баланс між швидкою доставкою та якістю;

Пояснювати бізнесу, чому іноді не варто поспішати з новими фічами.

Перевірка дублювання коду

Цей показник є важливим для контролю під час розробки програмного забезпечення на замовлення. Код, який містить велику кількість повторюваних фрагментів, вважається низькоякісним:

він займає більше рядків;

вимагає більше часу на обробку;

уповільнює роботу програмного забезпечення.

Іноді код дублюють свідомо — щоб спростити процес розробки, але частіше це випадкові й небажані повтори.

□ Навіщо це вимірювати:

Виявляти неоптимальні ділянки коду;

Зменшити розмір і складність проєкту;

Підвищити продуктивність програми;

Спростити тестування, підтримку та рефакторинг.

Користувацькі метрики

показники, які допомагають оцінити взаємодію користувачів з продуктом, рівень їхньої задоволеності, залученості та цінність продукту з точки зору кінцевого споживача.

CSAT

Оцінка задоволеності користувача CSAT (Customer Satisfaction Score) дозволяє оцінити якість конкретної взаємодії користувача з сервісом. Оцінюється за допомогою опитування. Наприклад, просять оцінити зручність навігації, відстеження доставки або зрозумілість програми лояльності — обираючи один із варіантів: «добре», «погано» або «нейтрально». □ Результат опитування не завжди відображає загальну лояльність користувача, адже йому могла сподобатися лише одна конкретна послуга, а решта — ні. Проте CSAT дозволяє швидко оцінювати окремі складові сервісу, з якими взаємодіяли користувачі.

□ CSAT важливий для:

Оцінки ключової функціональності застосунку. Наприклад, для інтернет-магазину важливо знати, чи задоволені користувачі каталогом товарів або системою оплати.

Аналізу «вразливих» або нових елементів. Якщо йдеться про інноваційні рішення або незвичні функції для конкретної аудиторії.

Перевірки ефективності редизайнуЩоб зрозуміти, чи вдалося покращити конкретні елементи інтерфейсу чи взаємодії з сервісом.

NPS

Net Promoter Score (NPS) — Індекс споживчої лояльності.

Метрика NPS розраховується на основі відповіді на запитання: «Наскільки ймовірно, що ви порекомендуєте наш сервіс друзям чи знайомим?» Зазвичай пропонується оцінити ймовірність за десятибальною шкалою — від 0 до 10.

□ На відміну від CSAT, яка показує задоволення від конкретної взаємодії в певний момент, NPS відображає загальну лояльність до продукту за весь період його використання.

□ Що показує NPS: Скільки користувачів — активні прихильники продукту та готові його рекомендувати;

Скільки — нейтральні, ще не визначилися з враженнями;

Скільки — незадоволені користувачі, які мають негативний досвід

□ Групи користувачів за шкалою NPS: 9–10 балів — Промоутери (лояльні, рекомендуючі); 7–8 балів — Нейтралі (задоволені, але не в захваті); 0–6 балів — Критики (незадоволені, можуть відмовитися від продукту або залишити негативні відгуки).

Feature Adoption Rate

Feature Adoption Rate — Рівень впровадження функціональності.

Feature Adoption Rate — це метрика, яка показує, який відсоток користувачів почали активно використовувати певну функцію після її запуску або оновлення. Вона допомагає зрозуміти, наскільки нова або ключова функціональність є корисною, помітною і зручною для аудиторії.

□ Навіщо відстежувати: Визначити, чи дійсно нова функція вирішує проблему користувача;

Порівнювати ефективність різних релізів чи А/Б-тестів;

Планувати оновлення, навчання або повідомлення в інтерфейсі для покращення впровадження.

User Retention Rate

Коефіцієнт утримання користувачів. Програми створюються для того, щоб ними користувалися регулярно, тому показник повернення користувачів є важливим для бізнесу. Цей показник дає уявлення про те, наскільки ефективно продукт утримує свою аудиторію після першого використання.

□ Як розраховується: Коефіцієнт утримання вимірюється як відсоток користувачів, які повернулися до продукту через певний період після першого використання (наприклад, через 7, 30 або 90 днів).

□ Навіщо відстежувати:

Оцінка задоволеності: високе значення Retention Rate свідчить про те, що користувачі задоволені продуктом і готові ним користуватися регулярно.

Виявлення проблем з продуктом: низький коефіцієнт утримання може вказувати на проблеми у взаємодії з користувачами, на відсутність цінності продукту.

Планування вдосконалень: допомагає зрозуміти, що потрібно покращити, щоб користувачі поверталися.

Conversion Rate

Коефіцієнт конверсії Метрика Conversion Rate показує, скільки людей серед відвідувачів додатка або вебсайту виконали цільову дію. Це може бути підписка, покупка, клік на потрібну кнопку тощо. Цільову дію можна визначити відповідно до мети конкретного аналізу. Наприклад, якщо за день на сторінці було 200 осіб, а 20 з них клікнули на потрібну кнопку, то коефіцієнт конверсії складе 10%.

□ Мета цієї метрики: Conversion Rate допомагає зрозуміти, наскільки ефективними є заклики до дії) для вашої аудиторії. Якщо коефіцієнт конверсії низький, це може вказувати на:

Проблеми в маркетингу:

Ціна товару або послуги занадто висока.

Низька якість товару або відсутність позитивних відгуків.

Нецікавість або байдужість аудиторії до продукту, послуги чи контенту.

Проблеми в роботі сайту: Сторінка оплати завантажується занадто довго, і через це користувачі не доходять до кінця покупки.

Проблеми в дизайні: Кнопка або елемент для кліку можуть бути непомітні або не привертати увагу.

MTTR

MTTR (Mean Time to Recovery / Repair) — це метрика, яка показує середній час, необхідний для відновлення роботи системи або виправлення інциденту після його виникнення.

□ Що означає MTTR?

MTTR = Mean Time to Recovery або Mean Time to Repair — середній час, за який команда усуває збій або проблему та повертає систему до нормального стану.

Приклад: За тиждень було 4 інциденти. Витрачено 8 годин на відновлення всіх інцидентів.

$MTTR = 8 \text{ годин} / 4 \text{ інциденти} = 2 \text{ години}$. Це означає, що в середньому потрібно 2 години, щоб відновити систему після кожного збою.

□ Навіщо вимірювати MTTR?

Оцінити ефективність роботи технічної підтримки або DevOps-команди.

Визначити, наскільки швидко система повертається в роботу після збоїв.

Виявити вузькі місця в процесі реагування на проблеми.

Підвищити довіру користувачів і якість обслуговування, зменшуючи тривалість простоїв.

Бізнес-метрики

використовуються для вимірювання результативності бізнесу в цілому. Вони дозволяють визначити, наскільки ефективно працює компанія в досягненні своїх стратегічних цілей. Зазвичай бізнес-метрики спрямовані на оцінку фінансових результатів, залучення та утримання клієнтів, а також ефективності операційних процесів.

Time to Market

- Time to Market (TTM) — це метрика, яка вимірює час, необхідний для розробки продукту або функціональності від ідеї до її виведення на ринок.
- Вона важлива для оцінки ефективності процесу розробки, а також для розуміння, скільки часу займає розробка і запуск нового продукту або функції. Чим менший час — тим конкурентніший продукт
- Time to Market вимірюється як час між двома ключовими подіями: Початок розробки — це момент, коли команда починає працювати над продуктом або функцією, тобто розпочинається розробка. Запуск на ринок — момент, коли продукт або нова функція стають доступними для користувачів (наприклад, публікація додатка в магазині додатків або випуск оновлення).

- ☐ **Чому це важливо?**
- **Конкурентоспроможність:** Виведення продукту на ринок швидше дозволяє компанії швидше реагувати на зміни попиту або нові можливості на ринку, а також зайняти лідируючі позиції.
- **Покращення планування:** Це дозволяє компанії оцінити, скільки часу займають розробка нових продуктів і як можна оптимізувати цей процес.
- **Адаптація до змін:** У деяких випадках на ринку можуть швидко змінюватися потреби або умови, тому швидкий **Time to Market** дозволяє компанії залишатися актуальною.

Метрика	Опис	Етап розробки	Відповідальні
Velocity	Швидкість виконання завдань у спринті	Розробка	Менеджери проектів
Lead Time	Час від постановки до виконання задачі	Розробка	DevOps, Менеджери
Cycle Time	Час від початку до завершення роботи над задачами	Розробка	Команда розробки
Burndown Chart	Візуалізація прогресу у виконанні запланованих робіт	Розробка	Скрам-майстри
Bug Rate	Частота виникнення помилок у коді	Передреліз	QA-інженери
Code Churn	Частота змін у кодовій базі	Розробка	Команда розробки
Deployment Frequency	Частота розгортання оновлень	Розробка	DevOps-інженери
Mean Time to Recovery (MTTR)	Час відновлення після збоїв	Постреліз	DevOps, SRE-команди

Customer Satisfaction (CSAT)	Рівень задоволеності користувачів продуктом	Постреліз	Продакт-менеджери
Net Promoter Score (NPS)	Індекс лояльності користувачів	Постреліз	Продакт-менеджери
Feature Adoption Rate	Рівень впровадження нових функцій	Постреліз	Продакт-менеджери
Time to Market	Час виходу продукту на ринок	Усі етапи	Менеджери, Бізнес
Escaped Defects	Помилки, виявлені користувачами після релізу	Постреліз	QA, Продакт-менеджери
Technical Debt	Обсяг технічного боргу у проєкті	Розробка, Постреліз	Архітектори, Розробники
User Retention Rate	Утримання користувачів після першої взаємодії	Постреліз	Продакт-менеджери

Висновок

- Контроль метрик допомагає не лише створити продукт, а й забезпечити його успіх на ринку.
- Регулярний моніторинг дозволяє приймати обґрунтовані рішення на кожному етапі розробки програмного продукту.